

Diseño de Sistemas de información

Profesores:

- Ing. Santiago Marnetto
- Ing. Impini Cristian

Ingeniería de software

La ingeniería de software es una tecnología con varias capas.



Capas de la ingeniería de software

Modelos

- Un modelo es una simplificación de la realidad creada para comprender mejor el sistema que se está creando.

- permiten describir, analizar, diseñar y documentar el sistema en desarrollo a lo largo de todo su ciclo de vida.

Modelo de Casos de Uso

- Proporciona un medio para capturar los requerimientos funcionales de un sistema, enfocándose en el punto de vista del usuario.

- Los casos de uso son las funciones que proporciona un sistema en beneficio de sus usuarios, de esta manera, cada usuario utiliza uno o más casos de uso para llevar a cabo su trabajo.

Modelo de Análisis

- Es una especificación detallada de los requerimientos y actúa como una primera aproximación al diseño del sistema.

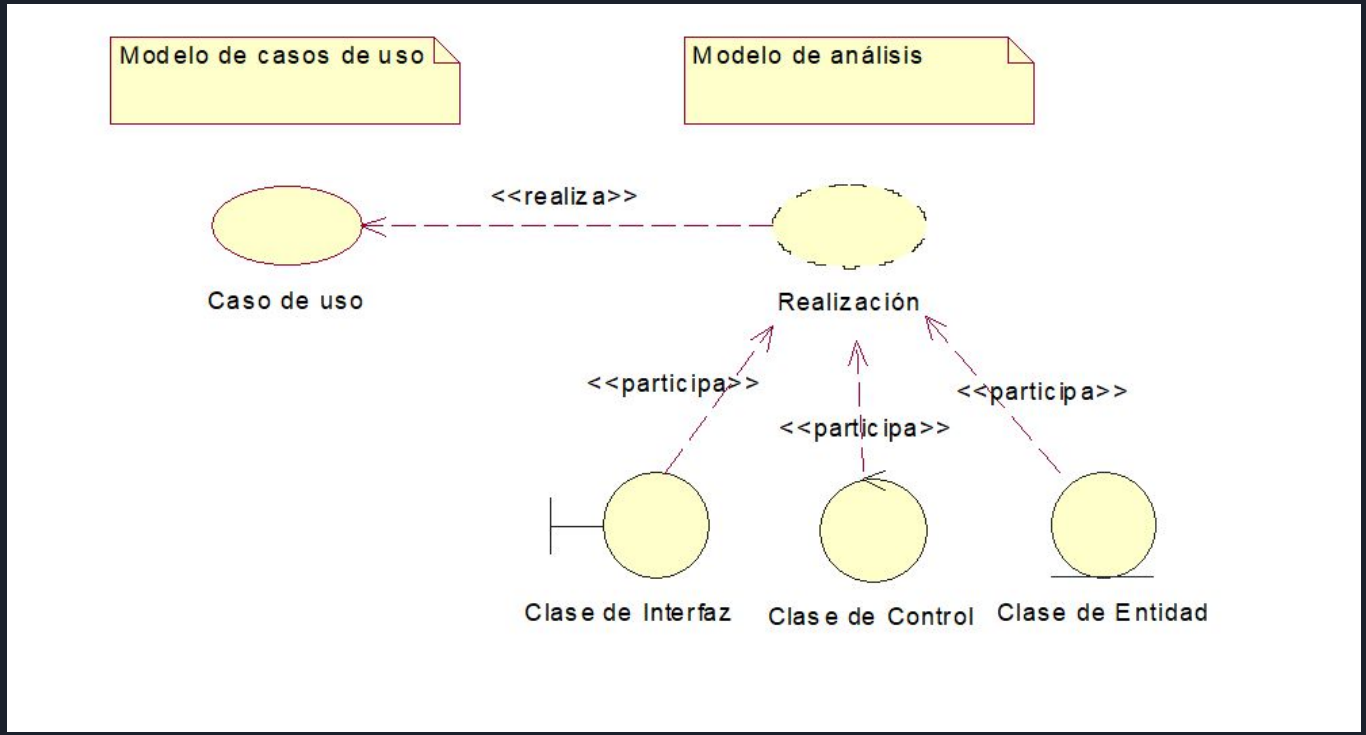
- Los casos de uso son las funciones que proporciona un sistema en beneficio de sus usuarios, de esta manera, cada usuario utiliza uno o más casos de uso para llevar a cabo su trabajo.

Modelo de Análisis

- Enfoca el análisis del sistema bajo tres perspectivas:
  - la interfaz entre el sistema y sus actores
  - la información utilizada en el sistema
  - la lógica de control del sistema

- Para cada caso de uso se efectúa una realización que muestra COMO se lleva a cabo la funcionalidad del caso de uso.

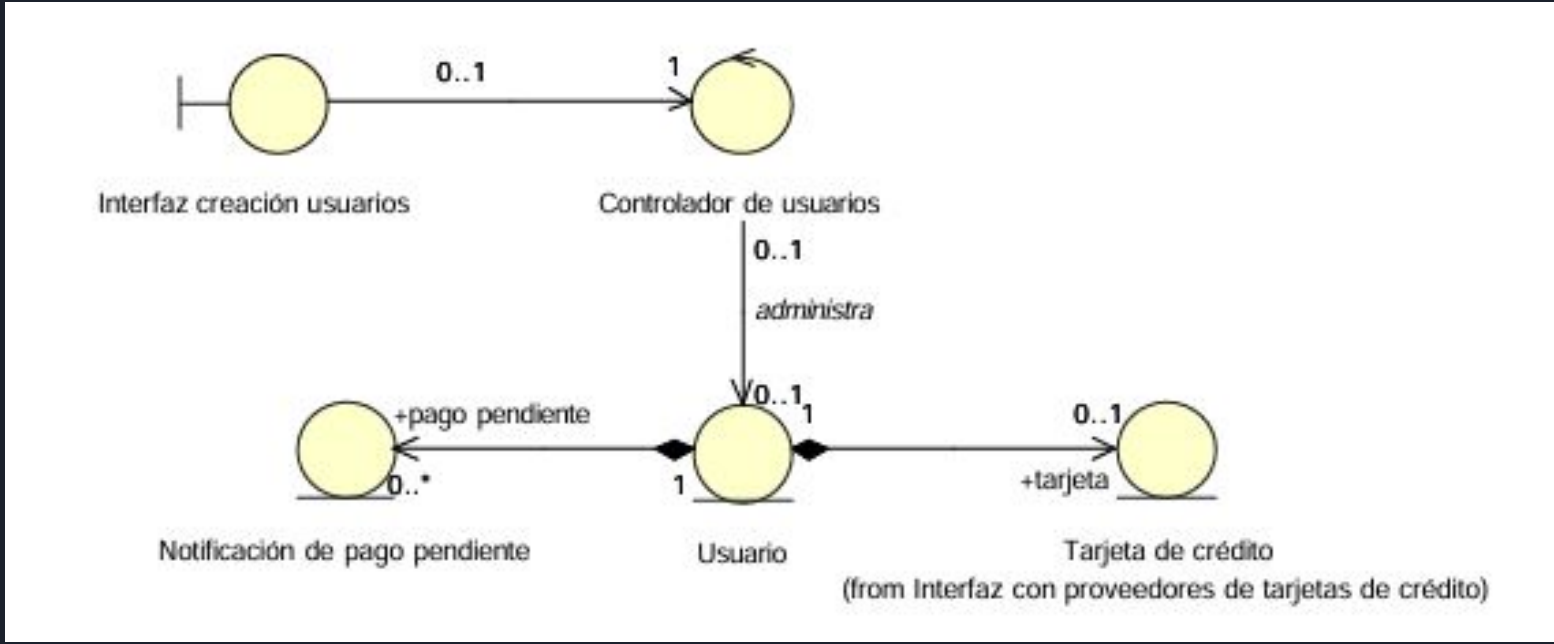
Modelo de Análisis



Modelo de Análisis

- Las responsabilidades de un objeto están dadas básicamente por las acciones que el objeto puede llevar a cabo y el conocimiento que el mismo mantiene y provee a otros objetos.

Modelo de Análisis



Modelo de casos de uso vs análisis

Escrito en el lenguaje del cliente	Descripto en el lenguaje de los desarrolladores
Vista externa del sistema.	Vista interna del sistema.
Estructurado en casos de uso. Da estructura a la vista externa del sistema.	Estructurado con clases estereotipadas (clases de análisis). Da estructura a la vista interna del sistema.
Usado como un “contrato” entre el cliente y los desarrolladores sobre lo que el sistema debe y no debe hacer.	Usado por los desarrolladores para entender cómo el sistema debería ser diseñado e implementado.

Modelo de casos de uso vs análisis

Puede haber redundancias e inconsistencias entre requerimientos.	No debería haber redundancias e inconsistencias entre requerimientos
Captura la funcionalidad del sistema.	Define cómo realizar la funcionalidad dentro del sistema, funciona como una primera aproximación al diseño.
Define casos de uso para que sean analizados en el modelo de análisis.	Define realizaciones de casos de uso, cada una de ellas representa el análisis de un caso de uso del modelo de casos de uso.

Modelo de diseño

- Se encarga de definir la forma y la arquitectura del sistema

- Convierte la visión abstracta dada por el “Modelo de análisis”, en una visión concreta de los componentes del sistema

Modelo de diseño

- Comprender en profundidad los requisitos no funcionales y las restricciones asociadas
- Comprender en profundidad los requisitos no funcionales y las restricciones asociadas
- Generar el punto de partida para las actividades posteriores de implementación del sistema
- Descomponer la implementación en partes más manejables
- Crear una abstracción de la implementación del sistema

Modelo de análisis vs diseño

Modelo conceptual	Modelo físico
Genérico	No genérico
Tiene 3 estereotipos conceptuales: Interfaz, Control y Entidad.	Puede tener cualquier número de estereotipos
Menos formal	Más formal

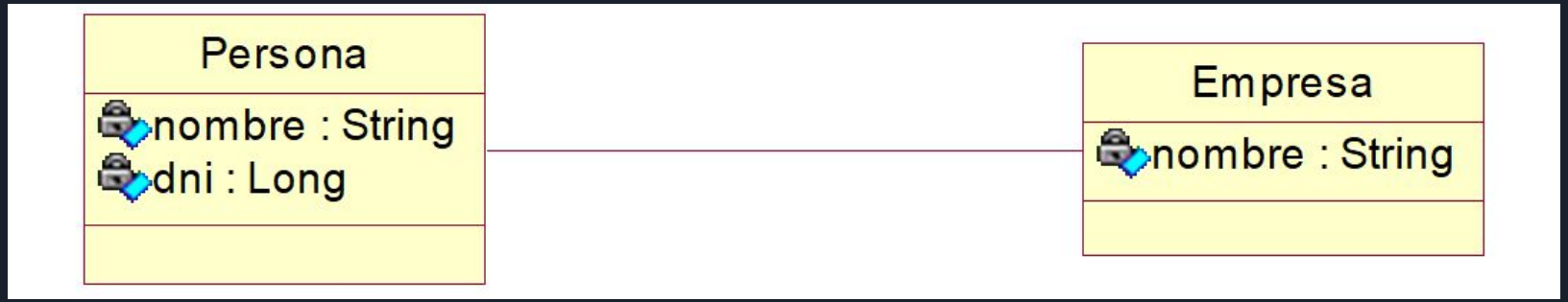
Modelo de análisis vs diseño

Menos capas	Más capas
Brinda un bosquejo del sistema	Es un manifiesto del sistema
Se crea principalmente como resultado de reuniones y talleres	Se crea principalmente como “programación visual”, en ingeniería de ida y vuelta con la implementación
Puede no mantenerse durante todo el ciclo de vida del software	Debe mantenerse durante todo el ciclo de vida

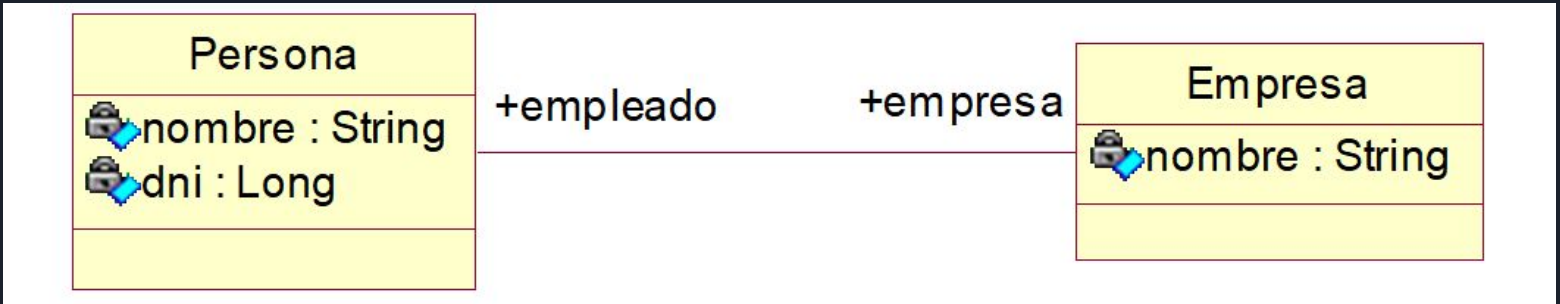
Clases de diseño



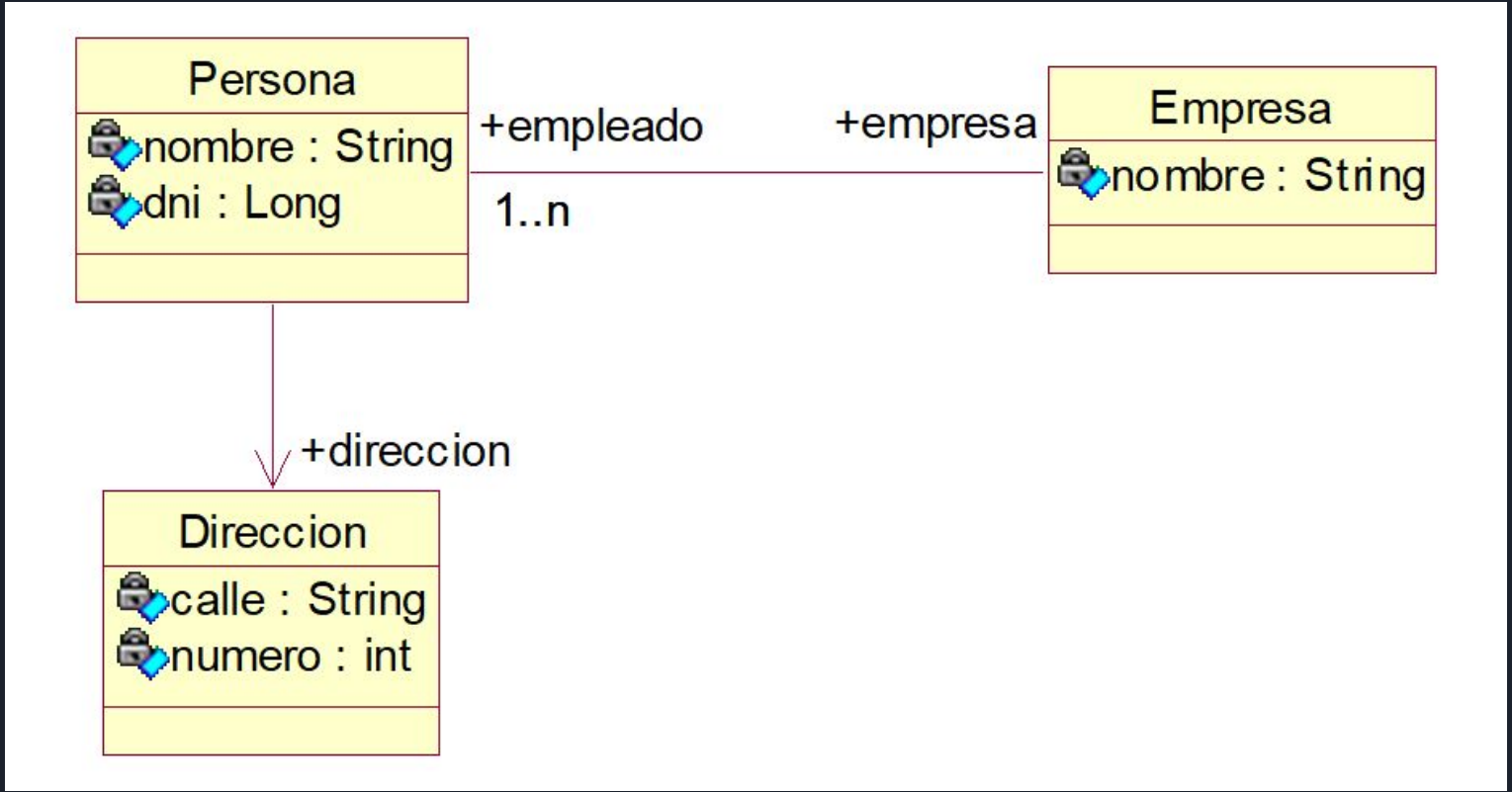
Relaciones



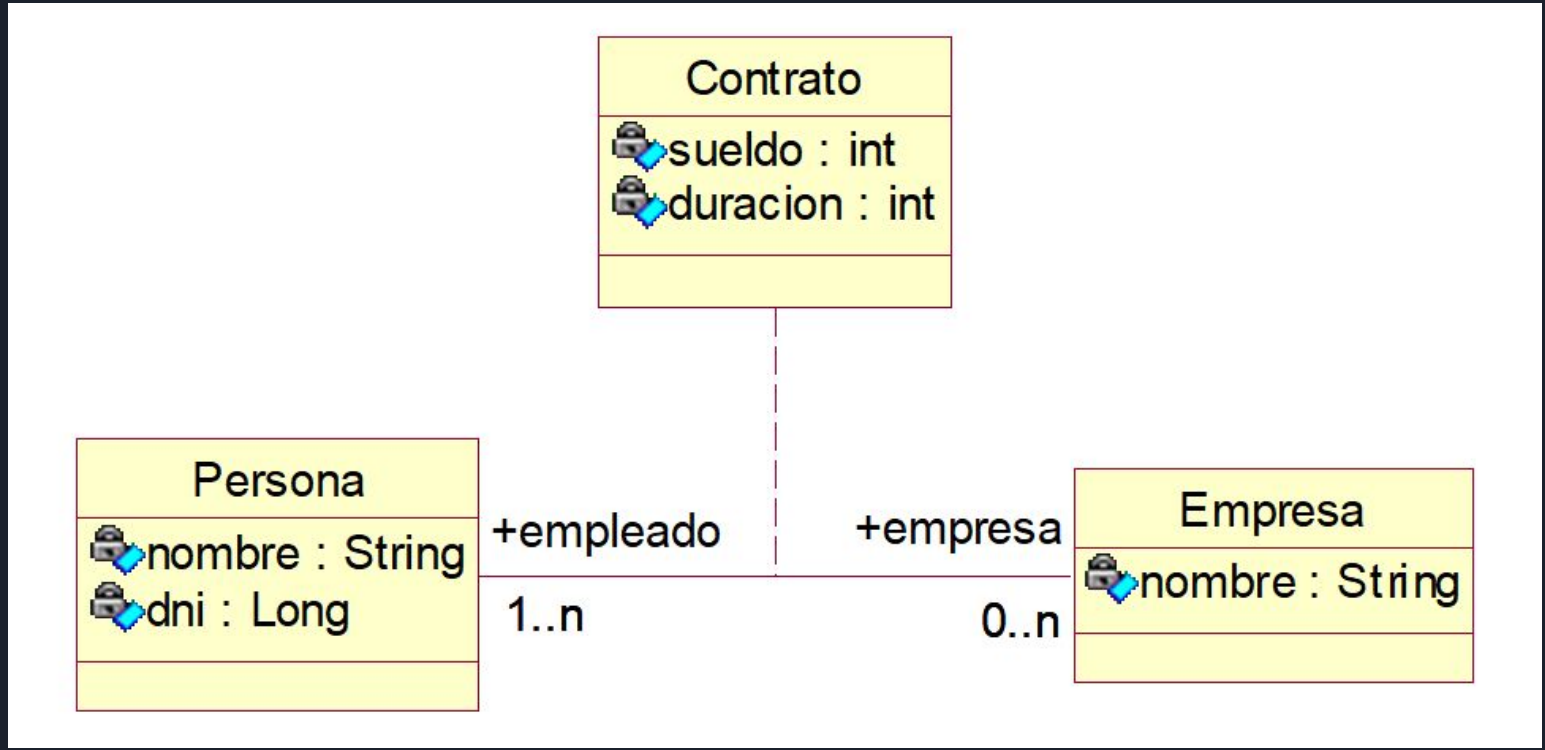
Relaciones – Roles y Nombre



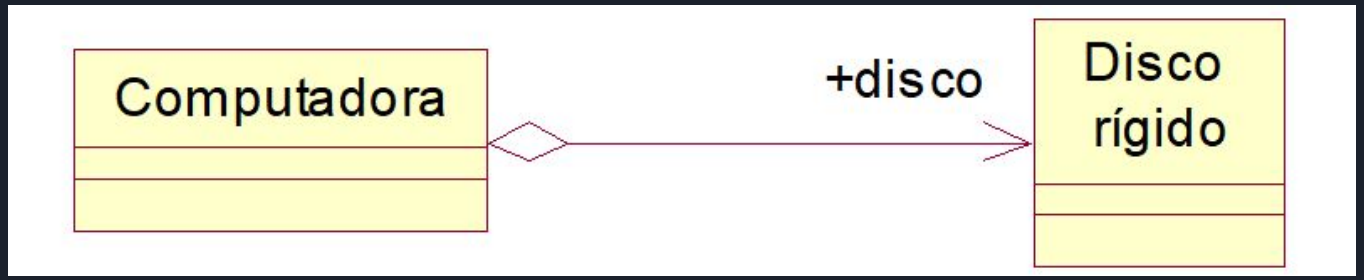
Relaciones


Persona
nombre : String
dni : Long
+empleado
1..n
+empresa
Empresa
nombre : String
+direccion
Direccion
calle : String
numero : int

Clases de asociación

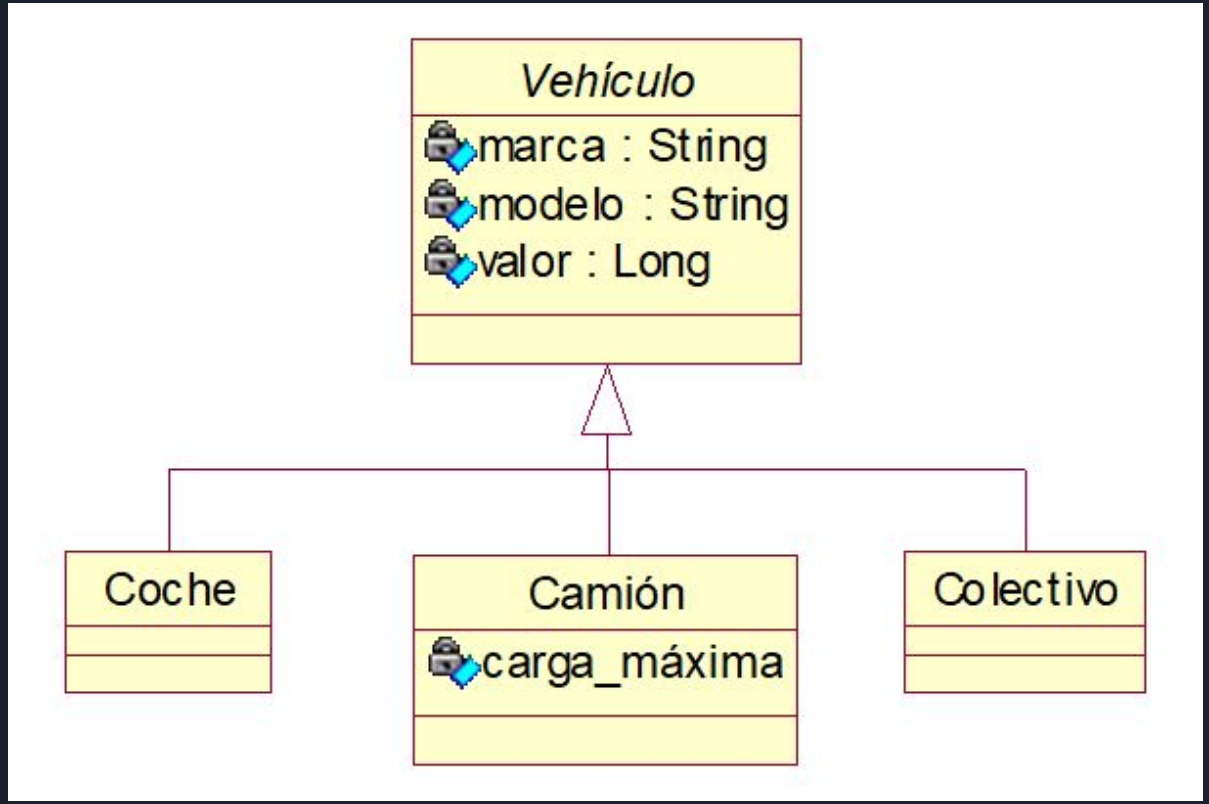


Agregaciones

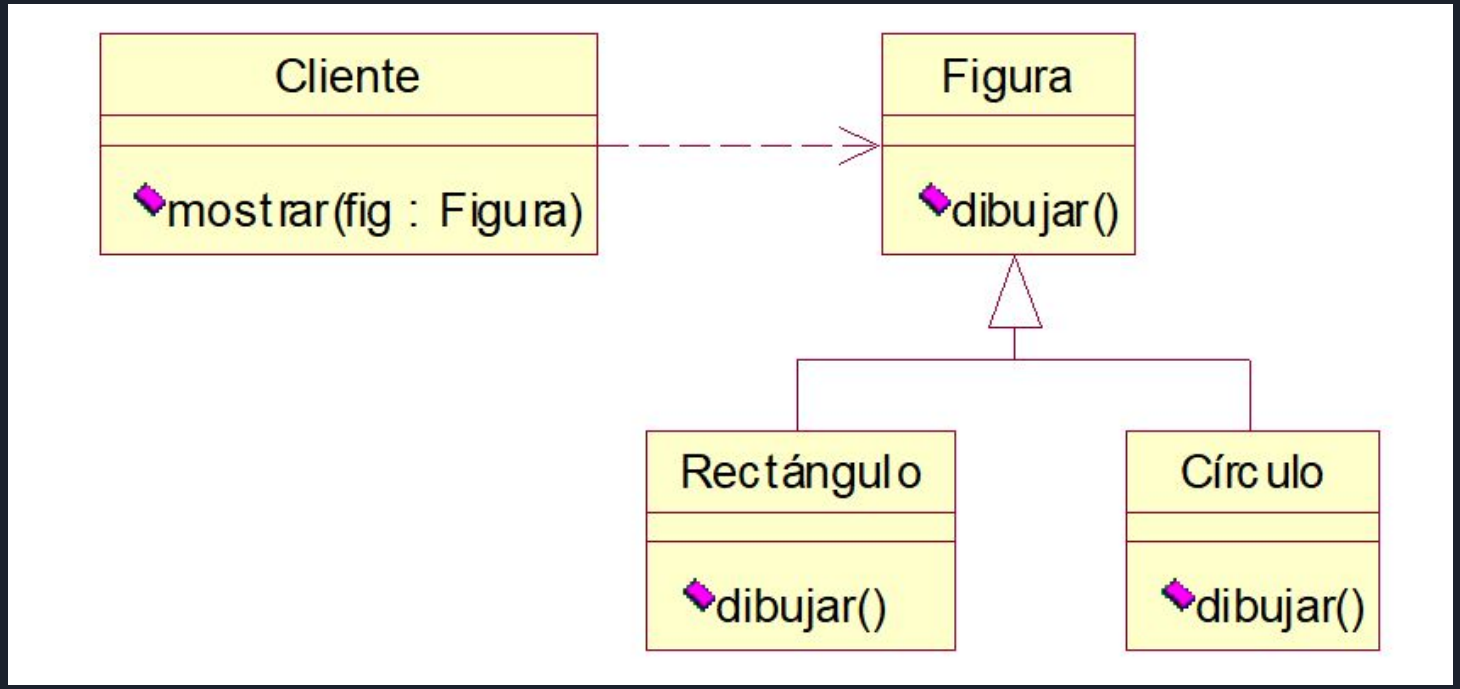




Generalizaciones



Clases Abstractas y dependencia



Clases abstractas

```
// Clase general Figura
abstract class Figura {
    public abstract void dibujar();
}
// Subclase Rectángulo
class Rectangulo extends Figura {
    @Override
    public void dibujar() {
        System.out.println("Dibujando un rectángulo.");
    }
}
// Subclase Circulo
class Circulo extends Figura {
    @Override
    public void dibujar() {
        System.out.println("Dibujando un círculo.");
    }
}
```

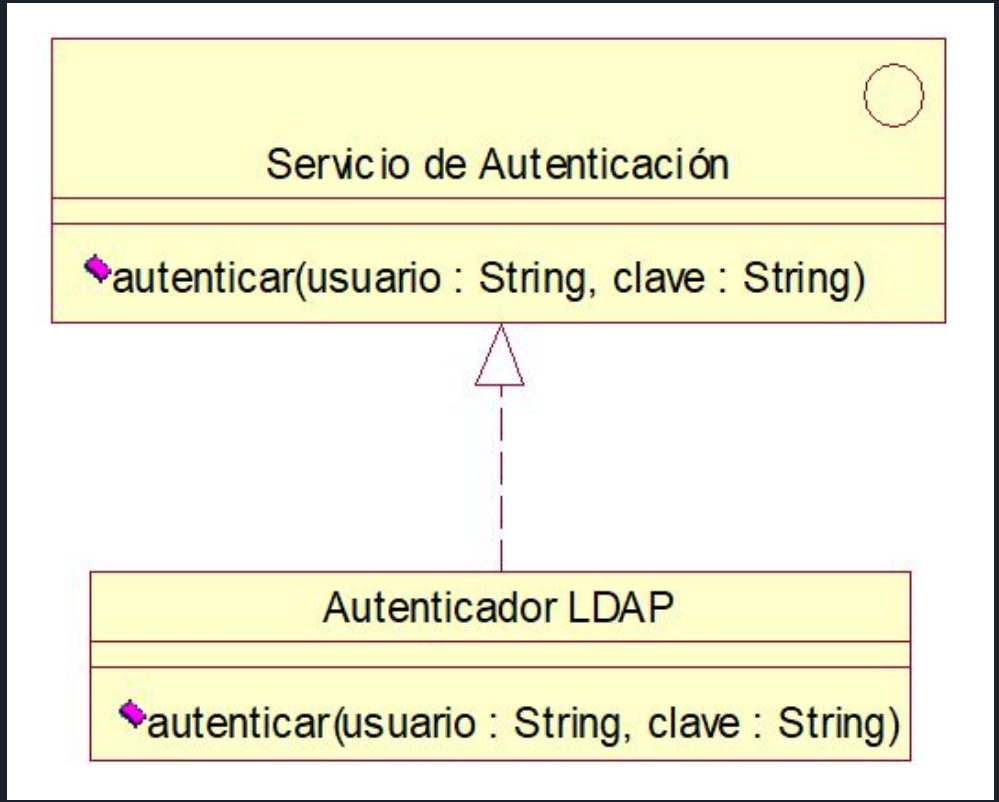
Clases abstractas

```
// Clase Cliente que usa Figura
class Cliente {
    public void mostrar(Figura fig) {
        System.out.println("Cliente va a mostrar la figura:");
        fig.dibujar(); // Polimorfismo en acción
    }
}
// Clase principal para probar
public class Main {
    public static void main(String[] args) {
        Cliente cliente = new Cliente();

        Figura rect = new Rectangulo();
        Figura circ = new Circulo();

        cliente.mostrar(rect);  // Salida: Dibujando un rectángulo.
        cliente.mostrar(circ);  // Salida: Dibujando un círculo.
    }
```

Interfaces


Servicio de Autenticación
autenticar(usuario : String, clave : String)
Autenticador LDAP
autenticar(usuario : String, clave : String)

Interfaces

```
/// Interfaz ServicioAutenticacion
public interface ServicioAutenticacion {
        boolean autenticar(String usuario, String contraseña);
}

// Implementación concreta: AutenticadorLDAP
public class AutenticadorLDAP implements ServicioAutenticacion {

        @Override
        public boolean autenticar(String usuario, String contraseña) {
        // Simulación de autenticación LDAP
        System.out.println("Autenticando con LDAP al usuario: " + usuario);

        // Validación simulada (en un caso real se consultaría un servidor LDAP)
        return "admin".equals(usuario) && "admin123".equals(contraseña);
        }
}
```

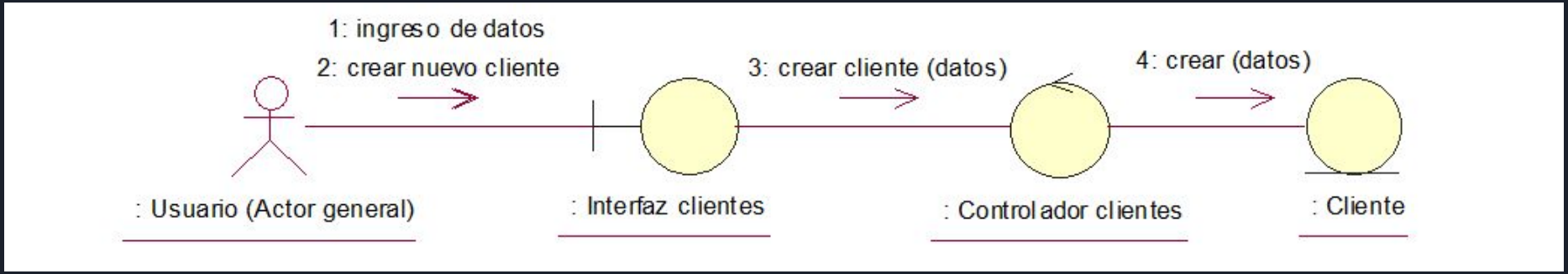
Interfaces

```
// Clase Cliente para probar la autenticación
public class Cliente {
        public static void main(String[] args) {
        ServicioAutenticacion servicio = new AutenticadorLDAP();

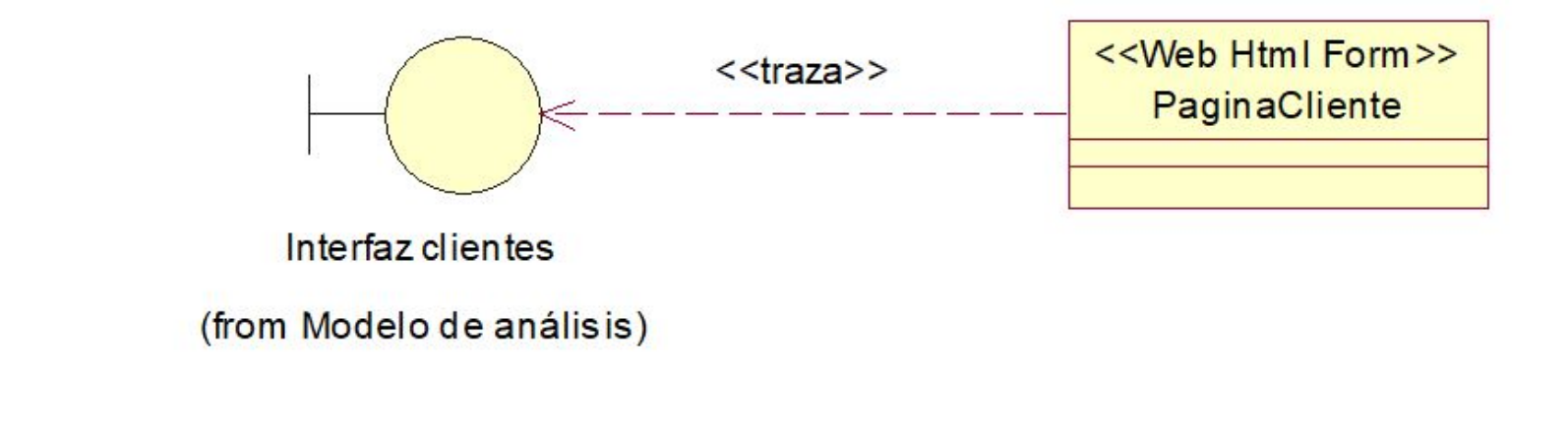
        boolean resultado = servicio.autenticar("admin", "admin123");

        if (resultado) {
        System.out.println("Autenticación exitosa.");
        } else {
        System.out.println("Autenticación fallida.");
        }
        }
}
```

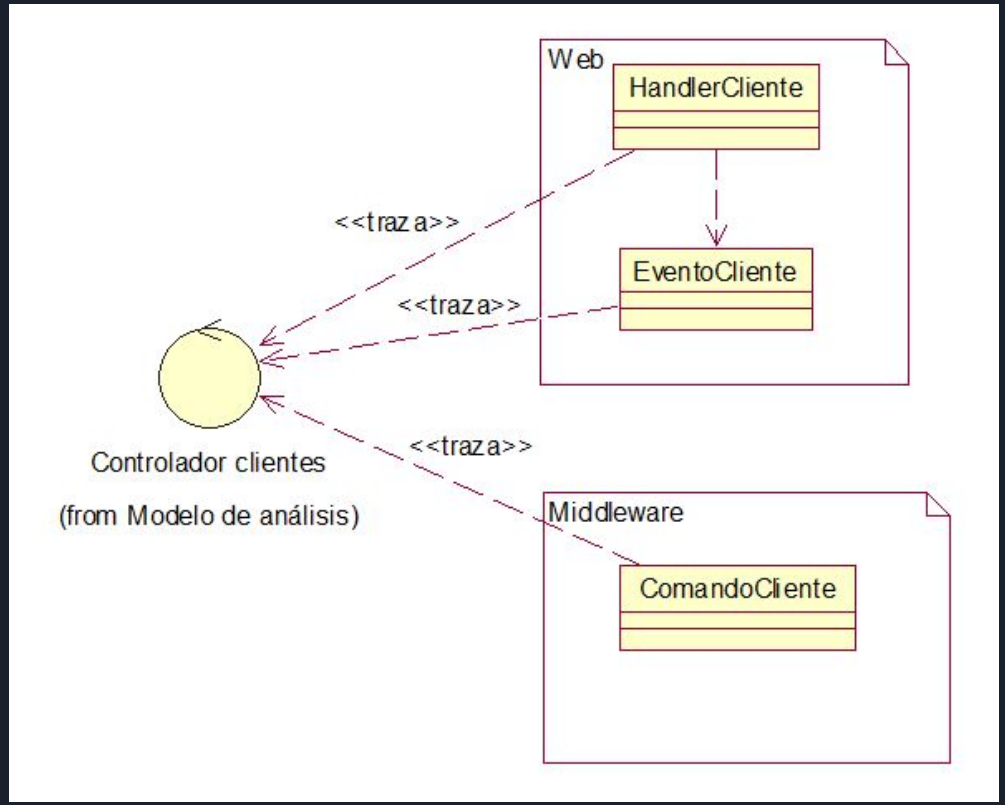
Identificación de clases



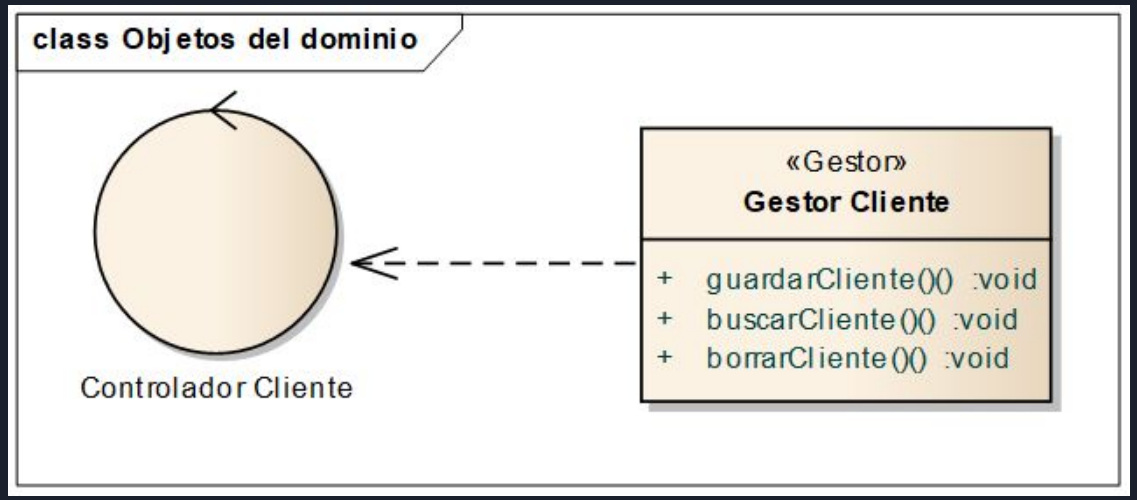
Identificación de clases



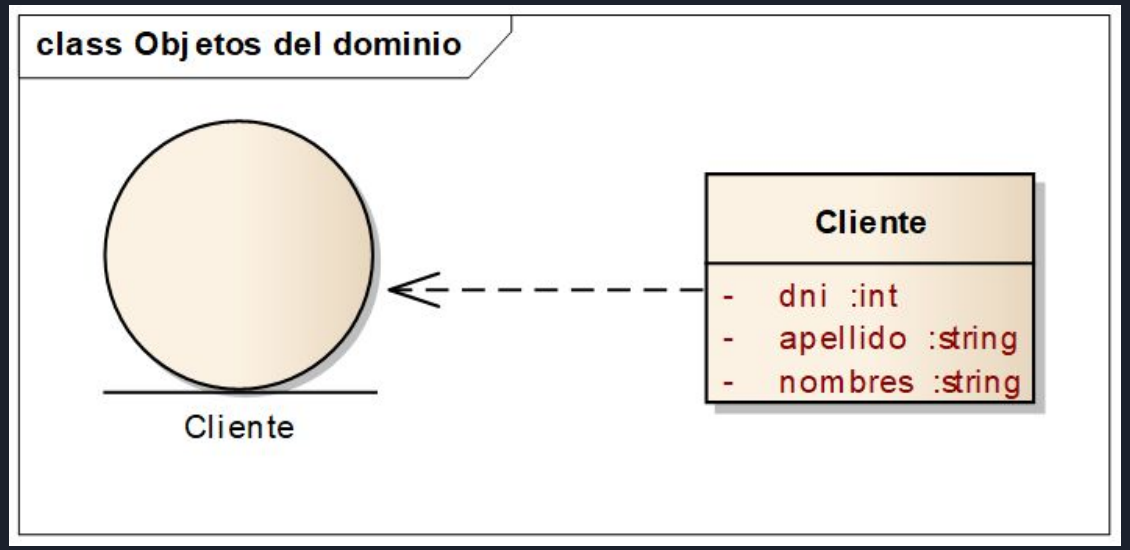
Identificación de clases



Identificación de clases



Identificación de clases



Identificación de clases

